A03b · Where the Promises Are Kept — Practice A2 · 9618 §19.2

Name: _____

Recursion is examined **twice** — explained on Paper 3, typed on Paper 4 — so this homework has both halves. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A03b_Practice.java`** (listed at the bottom). One set of numbers across both.

1. One call's paperwork. When a subroutine is called, the object code the compiler produced pushes a **frame** onto the stack. State **three** things that frame holds. [3]

1

2

3

2. The four-mark one. Explain how a **compiler** makes use of a stack when translating recursive programming code. [4]

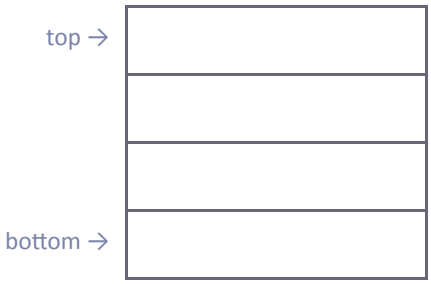
3. Draw the stack, not the table. This function counts how many digits a whole number has. [5]

```
FUNCTION Digits(N : INTEGER) RETURNS INTEGER
   IF N < 10 THEN
      RETURN 1
   ELSE
      RETURN 1 + Digits(N DIV 10)
   ENDIF
ENDFUNCTION
```

It is called as `Digits(4071)`.

(a) How many frames are on the call stack at the **deepest** moment? [1]

(b) Fill in the stack at that moment. Write the value of **N** held by each frame. [3]



(c) Which of those calls **finishes first**, and why? [1]

4. The other direction. Paper 4 asks you to rewrite recursion as iteration as often as the other way round. Here is a recursive routine: [4]

```
FUNCTION SumTo(N : INTEGER) RETURNS INTEGER
   IF N = 0 THEN
      RETURN 0
   ELSE
      RETURN N + SumTo(N - 1)
   ENDIF
ENDFUNCTION
```

(a) Write **pseudocode** for an iterative function **`SumToIterative(N)`** that returns the same answer. [3]

(b) State **one** thing the iterative version does not need, that the recursive one does. [1]

5. Why that structure? Explain the reasons why a **stack** is a suitable Abstract Data Type to implement recursion. [3]

6. Judgment. A program must count the lines in a log file that may hold **several million** lines. A student writes it recursively: one call per line. [2]

State whether this is a good use of recursion, and give a reason.

💻 **On the computer — `A03b_Practice.java`**

Fill in the four methods and run it — the judge is at the bottom of the file. *Both* sorts come from one real question: June 2024 Paper 42 asked for the recursive one, then for the same job as a loop.

7	**`RecursiveInsertion`** — insertion sort that calls itself	[4]
8	**`IterativeInsertion`** — the same sort, with loops	[4]
9	**`SumDigits`** — add a number's digits, recursively	[3]
10	🍦 **`InsertionDepth`** — *predict* how deep question 7 goes, before the judge measures it	[2]

Written 21 + code 13 = **34 marks**.

🕳 **Want to dig?** The lesson claimed a frame costs **96 bytes** and that the "limit on recursion" is really a division. Check it. Write a method that calls itself and counts, catch the **`StackOverflowError`**, and run it four times: **`java -Xint -Xss1m Runaway`**, then **`-Xss2m`**, **`-Xss4m`**, **`-Xss8m`**. Subtract two of your own numbers, divide by the extra bytes, and see what falls out. *(Drop **`-Xint`** and the numbers wobble — that is the compiler shrinking your frame mid-run.)*

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.